Análisis Semántico de Compiscript

Reporte de Implementación

Catedrático: Pablo Koch



REPOSITORIO GITHUB

Integrantes

Jose López - 23773
Pablo Cabrera - 231156
André Pivaral - 23574
Hansel López - 19026

6 de Septiembre de 2026

Índice

1. Introducción **2**

2. Arquitectura de la implementación **2**

2.1. Flujo general 2

2.2. Módulos 2

2.3. Manejo de errores 2

3. Analizador léxico y sintáctico **2**

4. Análisis semántico **3**

5. Tabla de símbolos **3**

6. Reglas semánticas implementadas **3**

6.1. Sistema de tipos 3

6.2. Ámbito 3

6.3. Funciones 4

6.4. Control de flujo 4

6.5. Clases y objetos 4

6.6. Listas 4

6.7. Reglas generales 4

7. Entorno de desarrollo **4**

8. Batería de pruebas **5**

9. Compilación y ejecución **5**

9.1. Requisitos 5

9.2. Compilación 5

9.3. Ejecución por línea de comandos 5

9.4. Ejecución del entorno gráfico 5

9.5. Ejecución de la batería de pruebas 6

10. Distribución del trabajo **6**

11. Conclusiones **6**

1. Introducción

Este reporte documenta la fase de análisis semántico de un compilador para Compiscript, un lenguaje basado en un subconjunto de TypeScript. El proyecto implementa el analizador sintáctico a partir de la gramática oficial en ANTLR y lo extiende con un recorrido semántico sobre el árbol sintáctico.

La implementación está escrita en Java. El análisis léxico y sintáctico usa ANTLR 4 con destino Java. El análisis semántico usa un visitor propio que extiende el visitor base generado. El proyecto incluye un entorno de escritorio construido con Swing y una batería de pruebas automatizada.

2. Arquitectura de la implementación

2.1. Flujo general

El compilador recibe un archivo fuente con extensión cps. El lexer y el parser de ANTLR construyen el árbol sintáctico. Si no se detectan errores léxicos ni sintácticos, el analizador semántico recorre ese árbol, consulta y construye la tabla de símbolos, y acumula errores y advertencias. Los resultados se muestran en el entorno gráfico o se imprimen y se exportan desde la línea de comandos.

2.2. Módulos

generated
: Lexer, parser y visitor base generados por ANTLR a partir de la gramática oficial.

semantic
: Sistema de tipos, símbolos, ámbitos y el analizador semántico.

viz
: Exportadores del árbol sintáctico a formato DOT y a imagen, y de la tabla de símbolos a texto Markdown.

gui
: Ventana principal de Swing, el ejecutor del pipeline y el objeto que transporta los resultados.

Main
: Punto de entrada con modo de línea de comandos y subcomando para lanzar el entorno gráfico.

2.3. Manejo de errores

El analizador semántico no lanza excepciones ante un error. Los errores se acumulan en una lista con categoría, mensaje, línea y columna, y el recorrido continúa. Cuando una expresión ya produjo un error de tipo se le asigna un tipo comodín compatible con cualquier otro, de modo que un solo error no genera una cadena de errores secundarios.

3. Analizador léxico y sintáctico

La gramática Compiscript.g4 es la provista y no fue modificada. ANTLR genera el lexer y el parser en Java durante la compilación.

Un listener recolector captura los errores del lexer con la categoría léxico y los errores del parser con la categoría sintáctico, en lugar de enviarlos a la salida de error estándar. Cada error registrado conserva su línea y su columna.

El árbol sintáctico se representa de forma visual. El recorrido del árbol emite nodos y aristas en formato DOT y una utilidad invoca el comando dot de Graphviz para producir la imagen.

4. Análisis semántico

El analizador semántico es un visitor que extiende el visitor base generado por ANTLR. El recorrido se realiza en dos pasadas.

1. Pre registro de clases y funciones de nivel superior. Permite la recursión y las referencias adelantadas.
2. Análisis completo. Verifica tipos, ámbitos, funciones, control de flujo, clases y listas, y llena la tabla de símbolos.

5. Tabla de símbolos

La tabla de símbolos maneja entornos anidados: global, función, clase y bloque. Cada entorno resuelve nombres de forma local y, si no los encuentra, sube por su entorno padre.

Cada símbolo almacena su nombre, su clase de símbolo, su tipo y la línea de su declaración. Los entornos no se destruyen al cerrarse, sino que se conservan para poder mostrar el estado completo de la tabla al final del análisis.

6. Reglas semánticas implementadas

6.1. Sistema de tipos

- Operaciones aritméticas sobre operandos de tipo entero o flotante, con coerción implícita de entero a flotante.
- Operaciones lógicas sobre operandos de tipo booleano.
- Compatibilidad de tipos en comparaciones.
- Asignaciones que respetan el tipo declarado de la variable.
- Inicialización obligatoria de las constantes en su declaración.
- Listas con elementos de tipo homogéneo.

6.2. Ámbito

- Resolución de nombres según ámbito local o global.
- Error por uso de una variable no declarada.
- Prohibición de redeclarar un identificador en el mismo ámbito.
- Un entorno nuevo por cada función, clase y bloque.

6.3. Funciones

- Verificación del número y del tipo de los argumentos en cada llamada, con correspondencia posicional.
- Verificación del tipo de retorno respecto al tipo declarado.
- Soporte para funciones recursivas.
- Soporte para funciones anidadas y closures, con captura de variables del entorno de definición.
- Detección de la redeclaración de funciones con el mismo nombre.

6.4. Control de flujo

- Las condiciones de las estructuras condicionales y de los bucles deben ser de tipo booleano.
- Las instrucciones de corte y de continuación solo se permiten dentro de un bucle.
- La instrucción de retorno solo se permite dentro del cuerpo de una función.

6.5. Clases y objetos

- Verificación de la existencia de los atributos y de los métodos accedidos con el operador punto.
- Verificación de la invocación correcta del constructor.
- Uso correcto de la referencia al objeto actual solo dentro de un método de clase.
- Herencia resuelta subiendo por la clase padre.

6.6. Listas

- Verificación del tipo de los elementos de una lista.
- El índice de acceso a una lista debe ser de tipo entero.

6.7. Reglas generales

- Detección de código muerto tras una instrucción de retorno, de corte o de continuación. Se reporta como advertencia.
- Validación de declaraciones duplicadas de variables, parámetros, funciones y atributos.

7. Entorno de desarrollo

El entorno gráfico está construido con Swing. La ventana tiene una barra superior, cuatro pestañas y una consola de texto.

Editor Área de texto donde se escribe o se carga el código fuente.

Errores
: Reporte agrupado por fase, con la línea y la columna de cada error.

Árbol Árbol sintáctico renderizado como imagen, con controles de acercamiento.

Símbolos
: Una tabla por cada entorno, con el nombre, la clase, el tipo y la línea de cada símbolo.

El pipeline se ejecuta en segundo plano. La interfaz cambia de color según la fase que falló, lo que permite distinguir de inmediato si el error fue léxico, sintáctico o semántico.

8. Batería de pruebas

La batería es un ejecutor propio que corre un conjunto de casos, uno por cada regla semántica, con un caso válido y un caso inválido cuando aplica. El ejecutor imprime el resultado de cada caso y termina con código de salida cero cuando todos pasan.

9. Compilación y ejecución

9.1. Requisitos

- Java 17 o una versión superior, con el compilador incluido.
- Graphviz de forma opcional, para renderizar el árbol como imagen.
- El archivo de ANTLR ya está incluido en el directorio lib.

9.2. Compilación

En Windows se ejecuta el script de compilación para Windows. En Linux y en Mac se ejecuta el script equivalente. El script genera el parser a partir de la gramática y compila todas las fuentes al directorio de salida.

```
build.bat
./build.sh
```

9.3. Ejecución por línea de comandos

Se llama la clase Main con el classpath del directorio de salida y del archivo de ANTLR, y la ruta del archivo fuente. Las banderas opcionales exportan el árbol y la tabla de símbolos. El código de salida es cero cuando no hay errores y uno en caso contrario.

```
java -cp "out;lib/antlr-4.13.2-complete.jar" Main archivo.cps
java -cp "out;lib/antlr-4.13.2-complete.jar" Main archivo.cps -dot -png -symbols -out output
```

9.4. Ejecución del entorno gráfico

Se llama la misma clase Main con el argumento del subcomando gráfico.

```
java -cp "out;lib/antlr-4.13.2-complete.jar" Main gui
```

9.5. Ejecución de la batería de pruebas

Se llama la clase del ejecutor de pruebas.

```
java -cp "out;lib/antlr-4.13.2-complete.jar" tests.TestRunner
```

10. Distribución del trabajo

El trabajo se repartió en cuatro contribuciones individuales, cada una con sus propios commits.

Jose López
: Infraestructura de ANTLR, integración de la gramática oficial, scripts de compilación y árbol sintáctico visual.

Hansel López
: Sistema de tipos y tabla de símbolos con entornos anidados y su exportador.

Pablo Cabrera
: Analizador semántico, ejecutor del pipeline y modo de línea de comandos.

André Pivaral
: Entorno gráfico de escritorio y batería de pruebas.

11. Conclusiones

La implementación cubre las reglas semánticas exigidas por el enunciado sobre un árbol sintáctico real generado por ANTLR. La tabla de símbolos sostiene entornos anidados a lo largo de todas las fases y su estado queda disponible para inspección. El entorno gráfico permite escribir, compilar y revisar el resultado de cada fase, y la batería de pruebas valida cada regla con casos válidos e inválidos.